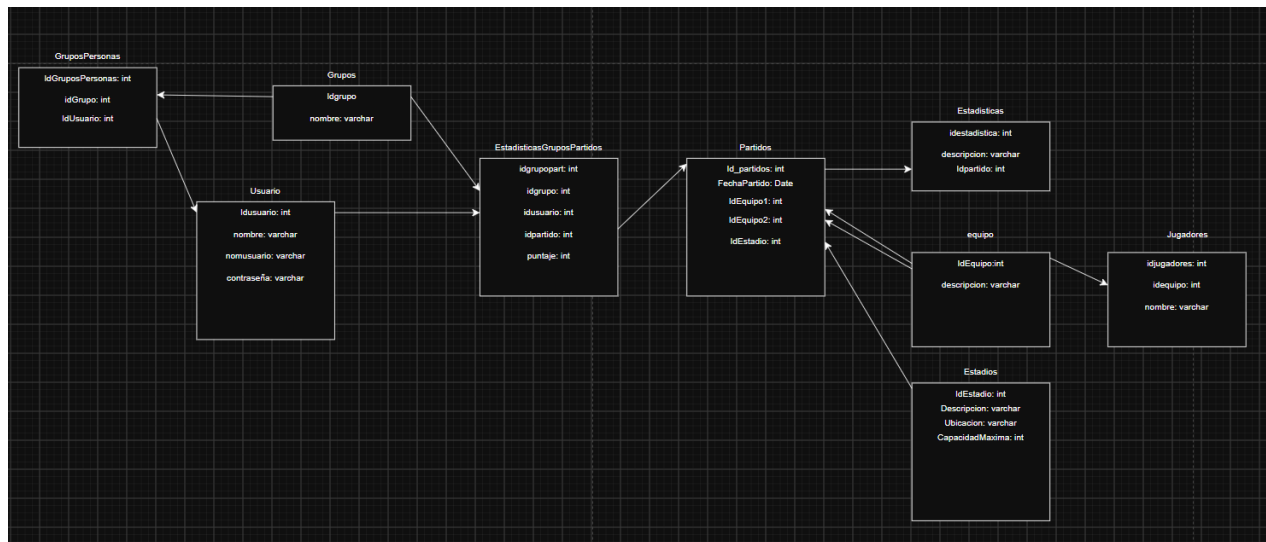


01 - Identificación de datos

01 - Identificación de datos

	Volumen	Crecimiento	Acceso
Partidos	127	Fijo	Lectura + Escritura
Selecciones	64	Fijo	Lectura
Estadios	20	Fijo	Lectura
Jugadores	1500+	Fijo	Lectura
Estadísticas grupos partidos	16 x cantidad de estadísticas por grupo	Crecimiento post partido	Lectura + Escritura
Grupos privados	Max 5 por usuario	Crecimiento masivo al inicio	Lectura + Escritura
Usuarios	2-3M simultáneos	Crecimiento masivo inicial con posibilidad de crecimiento continuo	Lectura + Escritura
Sesiones	2-3M simultáneos	Crecimiento irregular con picos masivos post partido	Lectura + Escritura
Estadísticas partido	1000+ por partido	Crecimiento durante partidos	Lectura + Escritura

Diagrama



02 - Problemas de SQL

01 - Problemas de SQL

Tipo Dato	Problema
Partidos	El partido sucede en tiempo real, eso significa que tendrá montones de read-write por segundo para ir ajustando los valores a lo que pasa en tiempo real. Puede ocasionar latencia en algo que debe ser inmediato
Selecciones	Al estar predefinidas y no requerir accesos constantes, ACID representa una pérdida en performance
estadio	Contiene pocos datos y estables, sql representa una desventaja por el consumo de incluir tablas y asegurar acid
jugadores	Misma lógica que anteriores, es igual de estático
Estadísticas grupos partidos	Muchos usuarios pueden actualizar resultados y posiciones al mismo tiempo, especialmente al finalizar un partido. Esto puede generar competencia por modificar las mismas filas, bloqueos, consultas agregadas costosas y demoras al recalcular las posiciones de cada grupo.
Usuarios	Durante un partido, montones de usuarios van a estar

	entrando y queriendo revisar su información, si no se maneja acorde puede haber latencia o delay
Grupos Privados	Al inicio del mundial, muchísimas personas van a anotarse, si se maneja con sql un volumen de grupos tan masivo puede generar cuelgues en los momentos más críticos de inscripciones
Estadísticas partido	Las estadísticas guardan momento a momento que está pasando en un partido, contando todos los partidos del mundial, esta tabla puede crecer drásticamente con el tiempo, volviéndose engorrosa de manejar con sql

03 - Propuesta de modelos NoSQL

01 - Propuesta de modelos NoSQL

Para cada tipo de dato identificado en las secciones anteriores, se propone a continuación el modelo NoSQL más adecuado, la razón de la elección, las ventajas frente a SQL y las alternativas que se consideraron antes de descartarlas.

Tipo de Dato	Modelo NoSQL Propuesto	Razón de la Elección	Ventajas vs SQL	Alternativas Consideradas
Partidos	Documental (MongoDB)	Cada partido puede modelarse como un único documento con su estado actual y un arreglo de eventos embebido, en vez de repartir la información en varias tablas relacionadas.	Se actualiza el documento entero sin necesidad de JOINS entre tablas; encaja mejor con el problema de read-write masivo en tiempo real detectado en la sección 2, porque no hay que tocar filas de otras tablas para reflejar un cambio.	Clave-valor (Redis): se pensó como caché del marcador en vivo, pero se descartó como modelo principal porque no sirve como fuente de verdad histórica del partido. Grafos (Neo4j): útil para modelar cruces del fixture, pero no resuelve el problema de latencia en vivo que se identificó como prioritario.
Selecciones	Documental (MongoDB)	Datos casi estáticos (64 selecciones) con	No hace falta pagar el costo de ACID para datos que casi	Clave-valor: se descartó porque perderíamos la estructura rica del

Tipo de Dato	Modelo NoSQL Propuesto	Razón de la Elección	Ventajas vs SQL	Alternativas Consideradas
		estructura anidada: plantel, cuerpo técnico, ranking.	no cambian; se lee todo en una sola consulta sin JOIN a la tabla de jugadores o cuerpo técnico.	documento (plantel completo analizado). Seguir en SQL: se consideró por el volumen bajísimo, pero se descartó por consistencia con el resto de la arquitectura.
Estadios	Documental (MongoDB)	Mismo criterio que Selecciones: pocos datos (20), estables, consultados con baja frecuencia.	Se evita crear y mantener una tabla relacional entera sólo para 20 filas fijas; el documento se puede embeber directamente en Partidos si conviene desnormalizar.	Seguir en SQL: se consideró por el volumen mínimo, pero se descartó para no mezclar dos paradigmas de persistencia distintos para datos de referencia similares.
Jugadores	Documental (MongoDB)	Mismo comportamiento estático que Selecciones y Estadios; se define además por consistencia con el resto de datos de referencia del torneo.	Documento por jugador con estadísticas históricas embebidas, sin tabla separada de stats; escala horizontalmente si se suman más torneos o temporadas más adelante.	Grafos (Neo4j): se pensó para relaciones jugador-selección-partido, pero se descartó porque no hay necesidad real de recorrer relaciones complejas tipo red social entre jugadores.
Estadísticas grupos partidos	Clave-valor (Redis)	El problema identificado en la sección 2 es la competencia por actualizar las mismas filas de posiciones al mismo tiempo, con bloqueos y consultas	Con estructuras tipo sortedset, leer y actualizar una tabla de posiciones es una operación casi instantánea, sin locks ni recálculo de agregados como en SQL.	Documental (MongoDB): se pensó un documento por grupo con la tabla de posiciones embebida, pero se descartó porque reescribir el documento entero en cada actualización es menos eficiente que un sortedset.

Tipo de Dato	Modelo NoSQL Propuesto	Razón de la Elección	Ventajas vs SQL	Alternativas Consideradas
		agregadas costosas.		
Grupos privados	Documental (MongoDB)	El problema detectado es el volumen masivo de altas al inicio del torneo; un documento por grupo con un arreglo de hasta 5 miembros resuelve esa carga.	Crear un grupo es insertar un único documento, sin tocar tablas de relación N a N ni bloquear otras filas; escala horizontalmente por sharding para absorber los picos de inscripción.	Clave-valor: se consideró (clave = id de grupo, valor = lista de miembros), pero se descartó porque documental permite indexar y filtrar por otros campos del grupo sin perder simplicidad.
Usuarios	Documental (MongoDB)	El problema es la cantidad de usuarios consultando su perfil durante el partido; un documento por usuario permite lecturas directas por id.	Esquema flexible para preferencias variables entre usuarios sin migraciones; no hace falta JOIN a tablas de configuración o preferencias.	Clave-valor puro: se descarto como modelo principal porque el perfil tiene estructura que conviene poder filtrar, no es solo un blob; queda como candidato para una capa de caché complementaria en momentos de pico.
Sesiones	Clave-valor (Redis)	Las sesiones son el caso clásico de clave-valor: token como clave, datos de sesión como valor, con necesidad de expiración automática.	Acceso en microsegundos por clave, soporta millones de sesiones simultáneas en memoria, y expira solas via TTL sin jobs de limpieza como haría falta en SQL.	Documental: se descartó porque las sesiones no necesitan estructura anidada ni consultas complejas, solo lectura/escritura rápida por clave; clave-valor es más simple y económico para este caso.
Estadísticas partido	Columnar (Cassandra)	El problema identificado es el crecimiento drástico momento a	Pensado para escrituras masivas y sostenidas (posesión, pases, tiros cada segundo)	Series temporales (InfluxDB): también encaja muy bien con datos momento a momento y fue

Tipo de Dato	Modelo NoSQL Propuesto	Razón de la Elección	Ventajas vs SQL	Alternativas Consideradas
		momento durante todos los partidos del mundial, que vuelve la tabla engorrosa en SQL.	sin degradarse, particionando naturalmente por partido y por tiempo; no exige ACID estricto en cada punto individual.	seriamente considerado; se optó por Cassandra como referencia de Big Data por ser el mismo caso de uso que Twitter visto en clase, aunque ambas opciones son técnicamente válidas.

Matriz de decisión (resumen)

Tipo de Dato	Modelo NoSQL	Motivo Principal
Partidos	Documental (MongoDB)	Update atómico del estado del partido en tiempo real
Selecciones	Documental (MongoDB)	Dato estático, estructura anidada, sin necesidad de ACID
Estadios	Documental (MongoDB)	Dato estático y de bajo volumen
Jugadores	Documental (MongoDB)	Dato estático con estadísticas embebidas
Estadísticas grupos partidos	Clave-valor (Redis)	Ranking que se actualiza en tiempo real sin bloqueos
Grupos privados	Documental (MongoDB)	Altas masivas al inicio, sin relaciones N a N complejas
Usuarios	Documental (MongoDB)	Perfil con estructura flexible y consulta por campos
Sesiones	Clave-valor (Redis)	Acceso rapido por clave con expiracion automatica (TTL)
Estadísticas partido	Columnar (Cassandra)	Escritura masiva y sostenida durante todo el torneo

04 - Conclusión

01 - Identificación de datos

Conclusión

A lo largo de este trabajo identificamos 9 tipos de datos del Fixture 2030, que se agrupan en tres familias bien distintas: datos de referencia casi fijos y de bajo volumen (Selecciones, Estadios, Jugadores), datos que crecen o se actualizan de forma masiva y en tiempo real durante los partidos (Partidos, Estadísticas grupos partidos, Estadísticas partido), y datos ligados directamente a la cantidad de usuarios conectados (Usuarios, Sesiones, Grupos privados). En SQL, cada una de estas familias termina chocando con el mismo tipo de problema: overhead de ACID para datos que casi no cambian, bloqueos y consultas agregadas costosas cuando muchos usuarios escriben al mismo tiempo, y tablas que crecen sin control cuando se registra el detalle momento a momento de cada partido.

Por qué Fixture 2030 necesita NoSQL

Los números del proyecto son el argumento más fuerte: 2 a 3 millones de usuarios simultáneos, picos de más de 100.000 requests por segundo y un objetivo de 99.99% de uptime no son compatibles con una base relacional única escalando verticalmente. SQL nos garantiza consistencia fuerte (ACID), pero esa garantía se paga con locks y JOINS que se vuelven más lentos justo cuando más tráfico hay, como en el entretiempo de un partido masivo. Los distintos modelos NoSQL propuestos en la sección 3 (documental, clave-valor y columnar) nos dejan relajar la consistencia donde no la necesitamos, como el número exacto de likes en un comentario, para ganar la disponibilidad y la velocidad de escritura que si necesitamos. Como se vio en clase con Twitter, Amazon y Netflix, ninguna de estas empresas usa un solo modelo de base de datos: la arquitectura políglota, combinando varios modelos NoSQL según el tipo de dato, es el camino más realista también para el Fixture 2030.

Proximos pasos

Este análisis es la base teórica sobre la que se va a construir el resto del cuatrimestre: en las próximas clases vamos a refinar la matriz de decisión de la sección 3 con el resto de la cátedra, y posteriormente comenzar la implementación concreta. Ahí vamos a poder confirmar en la práctica si las elecciones que hicimos en este documento (sobre todo las que dejamos anotadas como alternativas descartadas) siguen siendo las más apropiadas o si conviene ajustarlas.